

Chapter 1

Basic Concepts and Preliminaries

based on the following textbooks:

P. Tripathy and K. Naik. Software Evolution and Maintenance, Wiley,
2015

Outline

- Evolution vs. Maintenance
- Software Evolution Models and Processes
- Reengineering
- Legacy Systems
- Impact Analysis
- Refactoring
- Program Comprehension
- Software Reuse

Evolution versus Maintenance

- The concept of *software evolution* means a continual change from a lesser, simpler, or worse state to a higher or better state.
 - All activities carried out to effect changes in requirements are put under the category of evolution.
- The concept of *software maintenance* means preventing software from failing to deliver the intended functionalities by means of bug fixing.
 - All activities carried out *after* delivery of software are put under the category of maintenance.
 - Maintenance of software systems means fixing bugs but preserving their functionalities.

Evolution versus Maintenance

- Maintenance does not involve making major changes to the architecture and design of the system.
- Evolution of software systems means creating new but related designs from existing ones. The objectives include supporting new functionalities, making the system perform better, and making the system run on a different operating system.

Software Evolution Models and Processes

- The final phase of the Waterfall model is known as *maintenance*.
- Software maintenance should have its own *software maintenance life cycle (SMLC)* model.
 - SMLC models include three main features: understanding the code, modifying the code, and revalidating the code.
- Other models view software development as *iterative processes*.
 - Systems are constructed in builds, each of which is a refinement of requirements of the previous build.
 - A build is refined by considering feedback from users.
 - Maintenance and evolution activities do not exist as distinct phases.
- *Staged model of maintenance and evolution* is a software evolution model.
 - Its primary objective is to improve the understanding of how long-lived software evolves.
 - The model considers four sequential stages of the lifetime of a system:
 1. Initial development.
 2. Evolution.
 3. Servicing: here, the developers mainly focus on maintenance tasks, such as fixing bugs.
 4. Phaseout

Software Evolution Models and Processes

Software Maintenance Standards

- A well-defined process for software maintenance can be observed and measured, and thus improved.
- Process centric software maintenance is more of an engineering activity, with predictable time and effort constraints, and less of an art.
- Therefore, software maintenance standards have been formulated by ISO and IEEE.
 - The maintenance standard document from ISO is called [ISO/IEC 14764](#).
 - The maintenance standard document from IEEE is called [IEEE/EIA 1219](#).
- Both standards describe processes for managing and executing activities for maintenance. The activities are made up of tasks described with specific inputs, outputs, and actions.

Software Evolution Models and Processes

Software Configuration Management

- *Configuration management (CM)* is the discipline of managing changes in large systems.
 - Software configuration management (SCM) is the configuration management applied to software systems.
 - SCM is the means by which the process of software evolution is managed.
 - IEEE 1042 standard defines SCM as “the discipline of managing and controlling change in the evolution of software systems”.
 - Modern SCM tools provide advanced capabilities through which many different artifacts are managed.

Reengineering

- Reengineering is defined as the examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form.
- Software evolution can be seen as repeated software reengineering.
- Reengineering = Reverse engineering + Δ + Forward engineering
- *Reverse engineering* is the activity of defining a more abstract, and easier to understand, representation of the system.
- The core of reverse engineering is the process of examination, not a process of change, therefore it does not involve changing the software under examination.
- The third element “*forward engineering*,” is the traditional process of moving from high-level abstraction and logical, implementation-independent designs to the physical implementation of the system.
- The second element Δ captures alteration that is change of the system.

Legacy Systems

- A legacy system is an old program that continues to be used because it still meets the user's needs, in spite of the availability of newer technology or more efficient methods of performing the task.

Legacy Systems

- There are a number of options available to manage legacy systems. Typical solution include:
 - **Freeze**: The organization decides no further work on the legacy system should be performed.
 - **Outsource**: An organization may decide that supporting software is not core business, and may outsource it to a specialist organization offering this service.
 - **Carry on maintenance**: Despite all the problems of support, the organization decides to carry on maintenance for another period.
 - **Discard and redevelop**: Throw all the software away and redevelop the application once again from scratch.
 - **Wrap**: It is a black-box modernization technique – surrounds the legacy system with a software layer that hides the unwanted complexity of the existing data, individual programs, application systems, and interfaces with the new interfaces.
 - **Migrate**: Legacy system migration basically moves an existing, operational system to a new platform, retaining the legacy system's functionality and causing minimal disruption to the existing operational business environment as possible.

Impact Analysis

- *Impact analysis* is the task of estimating the parts of the software that can be affected if a proposed change request is made.
- Impact analysis techniques can be partitioned into two classes:
 - *Traceability analysis*: In this approach the high-level artifacts such as requirements, design, code and test cases related to the feature to be changed are identified. A model of inter-artifacts such that each artifact in one level links to other artifacts is constructed, which helps to locate pieces of design, code and test cases that need to be maintained.
 - *Dependency (or source-code) analysis*: Dependency analysis attempts to assess the affects of a change on the semantic dependencies between program entities. This is achieved by identifying the syntactic dependencies that may signal the presence of such semantic dependencies.
 - The two dependency-based impact analysis techniques are: *call graph-based* analysis and *dependency graph-based* analysis.

Impact Analysis

- *Ripple effect analysis* measures the impact, or how likely it is that a change to a particular module may cause problems in the rest of the program.
- *Change propagation* activities ensure that a change made in one component is propagated properly throughout the entire system.

Refactoring

- *Refactoring* is the process of making a change to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.
 - For non-object-oriented systems, it is equivalent to *restructuring*.
- Refactoring is achieved through the removal of duplicate code, simplification of code, and moving code to a different class, among others.
- Refactoring must preserve the “observable behavior” of the software system.
 - Regression testing is used to ensure that the system did not deviate from the original system during refactoring.
- Refactoring is done to improve the internal structure of a software system (improve its maintainability).

Program Comprehension

- *Program understanding or comprehension* is the task of building mental models of an underlying software system at various abstraction levels, ranging from models of the code itself to ones of the underlying application domain, for software maintenance, evolution and re-engineering purposes.
- A mental model describes a programmer's mental representation of the program to be comprehended.
- Program comprehension deals with the cognitive processes involved in constructing a mental model of the program.
- A common element of such cognitive models is generating hypotheses and investigating whether they hold or must be rejected.
 - Hypotheses are a way to understand code in an incremental manner.
 - After some understanding of the code, the programmer forms a hypothesis and verifies it by reading code.
 - By continuously formulating new hypotheses and verifying them, the programmer understands more and more code and in increasing details.

Software Reuse

- *Software reuse* is using existing artifacts or software knowledge during the construction of a new software system.
 - Reusable assets can be either reusable artifacts or software knowledge.
- There are four types of reusable artifacts:
 - *Data reuse*: involves a standardization of data formats.
 - *Architectural reuse*: consists of standardizing a set of design and programming conventions dealing with the logical organization of software.
 - *Design reuse*: deals with the reuse of abstract designs that do not include implementation details
 - *Program reuse*: deals with reusing executable code.

Software Reuse

- Reusability is a property of software assets, which indicates the degree to which the software can be reused.
- For a software component to be reused, it must exhibit the following characteristics: high cohesion, low coupling, adaptability, understandability, reliability, and portability.
- There are two advantages of reusing previously written code:
 - Better quality
 - Increased productivity

The End of Chapter 1